

Stage1:CNN Experiments

Experiment 1 — Basic CNN

Goal:

To establish a baseline accuracy before making any changes.

Changes:

- 3 Conv2D layers: **32** → **32** → **64**
- **12 epochs**

Results:

- **Final Training Accuracy: 96.60%**
- **Final Validation Accuracy: 67.18%**
- **Best Validation Accuracy: 67.42% (Epoch 11)**

Observation:

The large gap between training and validation accuracy indicates overfitting.

```
num_classes = 6

# Creating the model
model=tf.keras.Sequential([
    tf.keras.layers.Rescaling(1.0/255,input_shape=(128,128,3)),
    tf.keras.layers.Conv2D(32,3,activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(32,3,activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(64,3,activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128,activation='relu'),
    tf.keras.layers.Dense(num_classes,activation='softmax')
])

# Compiling the model.
model.compile(
    optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=["accuracy"]
)

#fitting the model
model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=12
)
```

```

Epoch 1/12
325/325 ————— 32s 99ms/step - accuracy: 0.4085 - loss: 1.4686 - val_accuracy: 0.4919 - val_loss: 1.3349
Epoch 2/12
325/325 ————— 34s 103ms/step - accuracy: 0.5341 - loss: 1.2286 - val_accuracy: 0.5388 - val_loss: 1.2604
Epoch 3/12
325/325 ————— 33s 101ms/step - accuracy: 0.6081 - loss: 1.0637 - val_accuracy: 0.5858 - val_loss: 1.1459
Epoch 4/12
325/325 ————— 32s 99ms/step - accuracy: 0.6753 - loss: 0.9052 - val_accuracy: 0.6022 - val_loss: 1.1639
Epoch 5/12
325/325 ————— 32s 97ms/step - accuracy: 0.7490 - loss: 0.7162 - val_accuracy: 0.6298 - val_loss: 1.2103
Epoch 6/12
325/325 ————— 32s 98ms/step - accuracy: 0.8109 - loss: 0.5442 - val_accuracy: 0.6291 - val_loss: 1.2550
Epoch 7/12
325/325 ————— 32s 97ms/step - accuracy: 0.8686 - loss: 0.3950 - val_accuracy: 0.6621 - val_loss: 1.5183
Epoch 8/12
325/325 ————— 32s 97ms/step - accuracy: 0.9045 - loss: 0.2781 - val_accuracy: 0.6601 - val_loss: 1.6297
Epoch 9/12
325/325 ————— 32s 100ms/step - accuracy: 0.9320 - loss: 0.2061 - val_accuracy: 0.6722 - val_loss: 1.9202
Epoch 10/12
325/325 ————— 32s 99ms/step - accuracy: 0.9492 - loss: 0.1562 - val_accuracy: 0.6731 - val_loss: 1.9881
Epoch 11/12
325/325 ————— 32s 98ms/step - accuracy: 0.9594 - loss: 0.1254 - val_accuracy: 0.6507 - val_loss: 2.2274
Epoch 12/12
325/325 ————— 32s 99ms/step - accuracy: 0.9676 - loss: 0.1072 - val_accuracy: 0.6830 - val_loss: 2.2856

```

Experiment 2 — Data Augmentation

Goal:

To reduce overfitting by adding data augmentation.

Changes:

- **Added Data Augmentation**
 - *Random Horizontal Flip*
 - *Random Rotation: 0.1*
 - *Random Zoom: 0.1*
- **3 Conv2D layers: 32 → 32 → 64**
- **12 epochs**
- **No Dropout**

Results:

- **Final Training Accuracy: 67.29%**
- **Final Validation Accuracy: 60.71%**
- **Best Validation Accuracy: 60.71% (Epoch 12)**

Observation:

Training and validation accuracy are closer, showing reduced overfitting, but validation accuracy decreased compared to Experiment 1.

```

65]: num_classes = 6
      #Data Augmentation
      data_augmentation=tf.keras.Sequential([
          tf.keras.layers.RandomFlip("horizontal",input_shape=(128,128,3)),
          tf.keras.layers.RandomRotation(0.1),
          tf.keras.layers.RandomZoom(0.1)
      ])
      # Creating the model
      model=tf.keras.Sequential([
          data_augmentation,
          tf.keras.layers.Rescaling(1.0/255),
          tf.keras.layers.Conv2D(32,3,activation='relu'),
          tf.keras.layers.MaxPooling2D(),
          tf.keras.layers.Conv2D(32,3,activation='relu'),
          tf.keras.layers.MaxPooling2D(),
          tf.keras.layers.Conv2D(64,3,activation='relu'),
          tf.keras.layers.MaxPooling2D(),
          tf.keras.layers.Flatten(),
          tf.keras.layers.Dense(128,activation='relu'),
          tf.keras.layers.Dense(num_classes,activation='softmax')
      ])

66]: # Compiling the model.
      model.compile(
          optimizer='adam',
          loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
          metrics=["accuracy"]
      )

[*]: #fitting the model
      model.fit(
          train_ds,
          validation_data=val_ds,
          epochs=12
      )

```

```

Epoch 1/12
325/325 ————— 36s 109ms/step - accuracy: 0.3753 - loss: 1.5355 - val_accuracy: 0.4517 - val_loss: 1.4271
Epoch 2/12
325/325 ————— 35s 107ms/step - accuracy: 0.4749 - loss: 1.3550 - val_accuracy: 0.4836 - val_loss: 1.3719
Epoch 3/12
325/325 ————— 36s 110ms/step - accuracy: 0.5113 - loss: 1.2813 - val_accuracy: 0.5213 - val_loss: 1.2728
Epoch 4/12
325/325 ————— 35s 108ms/step - accuracy: 0.5357 - loss: 1.2265 - val_accuracy: 0.5368 - val_loss: 1.2491
Epoch 5/12
325/325 ————— 35s 108ms/step - accuracy: 0.5666 - loss: 1.1722 - val_accuracy: 0.5153 - val_loss: 1.3676
Epoch 6/12
325/325 ————— 35s 107ms/step - accuracy: 0.5902 - loss: 1.1154 - val_accuracy: 0.5332 - val_loss: 1.3216
Epoch 7/12
325/325 ————— 34s 105ms/step - accuracy: 0.6121 - loss: 1.0672 - val_accuracy: 0.5642 - val_loss: 1.2160
Epoch 8/12
325/325 ————— 34s 105ms/step - accuracy: 0.6221 - loss: 1.0360 - val_accuracy: 0.5350 - val_loss: 1.4352
Epoch 9/12
325/325 ————— 35s 107ms/step - accuracy: 0.6352 - loss: 1.0046 - val_accuracy: 0.5786 - val_loss: 1.2146
Epoch 10/12
325/325 ————— 34s 105ms/step - accuracy: 0.6453 - loss: 0.9738 - val_accuracy: 0.5683 - val_loss: 1.3193
Epoch 11/12
325/325 ————— 34s 105ms/step - accuracy: 0.6583 - loss: 0.9414 - val_accuracy: 0.6026 - val_loss: 1.1546
Epoch 12/12
325/325 ————— 34s 105ms/step - accuracy: 0.6729 - loss: 0.9089 - val_accuracy: 0.6071 - val_loss: 1.1575

```

Experiment 3 — Dropout

Goal:

To reduce overfitting by adding Dropout.

Changes:

- Added Dropout (0.5)
- 3 Conv2D layers: 32 → 32 → 64

- **Dense layer: 128 neurons**
- **12 epochs**
- **No Data Augmentation**

Results:

- **Final Training Accuracy: 81.78%**
- **Final Validation Accuracy: 66.97%**
- **Best Validation Accuracy: 66.97% (Epoch 12)**

Observation:

The gap between training and validation accuracy was reduced compared to Experiment 1, while validation accuracy remained close to the baseline.

```
8): num_classes = 6

# Creating the model
model=tf.keras.Sequential([
    tf.keras.layers.Rescaling(1.0/255,input_shape=(128,128,3)),
    tf.keras.layers.Conv2D(32,3,activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(32,3,activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(64,3,activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128,activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(num_classes,activation='softmax')
])

9): # Compiling the model.
model.compile(
    optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=["accuracy"]
)

*): #fitting the model
model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=12
)
```

```

Epoch 1/12
325/325 ————— 34s 102ms/step - accuracy: 0.3535 - loss: 1.5727 - val_accuracy: 0.4432 - val_loss: 1.4095
Epoch 2/12
325/325 ————— 33s 101ms/step - accuracy: 0.4777 - loss: 1.3644 - val_accuracy: 0.4982 - val_loss: 1.3187
Epoch 3/12
325/325 ————— 32s 99ms/step - accuracy: 0.5233 - loss: 1.2544 - val_accuracy: 0.5501 - val_loss: 1.1972
Epoch 4/12
325/325 ————— 32s 98ms/step - accuracy: 0.5676 - loss: 1.1531 - val_accuracy: 0.5965 - val_loss: 1.1392
Epoch 5/12
325/325 ————— 32s 98ms/step - accuracy: 0.6190 - loss: 1.0371 - val_accuracy: 0.6192 - val_loss: 1.0845
Epoch 6/12
325/325 ————— 32s 99ms/step - accuracy: 0.6536 - loss: 0.9380 - val_accuracy: 0.6289 - val_loss: 1.1162
Epoch 7/12
325/325 ————— 63s 193ms/step - accuracy: 0.6922 - loss: 0.8477 - val_accuracy: 0.6405 - val_loss: 1.0646
Epoch 8/12
325/325 ————— 34s 103ms/step - accuracy: 0.7183 - loss: 0.7662 - val_accuracy: 0.6439 - val_loss: 1.0715
Epoch 9/12
325/325 ————— 32s 99ms/step - accuracy: 0.7488 - loss: 0.6790 - val_accuracy: 0.6619 - val_loss: 1.1046
Epoch 10/12
325/325 ————— 32s 98ms/step - accuracy: 0.7787 - loss: 0.6071 - val_accuracy: 0.6668 - val_loss: 1.1281
Epoch 11/12
325/325 ————— 33s 100ms/step - accuracy: 0.7989 - loss: 0.5450 - val_accuracy: 0.6693 - val_loss: 1.1816
Epoch 12/12
325/325 ————— 32s 98ms/step - accuracy: 0.8178 - loss: 0.4908 - val_accuracy: 0.6697 - val_loss: 1.2487

```

Experiment 4 — Data Augmentation + Dropout

Goal:

To combine Data Augmentation and Dropout to reduce overfitting.

Changes:

- **Added Data Augmentation**
 - *Random Horizontal Flip*
 - *Random Rotation: 0.1*
 - *Random Zoom: 0.1*
- **Added Dropout (0.5)**
- **3 Conv2D layers: 32 → 32 → 64**
- **12 epochs**

Results:

- **Final Training Accuracy: 58.74%**
- **Final Validation Accuracy: 58.73%**
- **Best Validation Accuracy: 58.76% (Epoch 11)**

Observation:

Training and validation accuracy were very close, but validation accuracy was lower than Experiments 1 and 3.

```

num_classes = 6
#Data Augmentation
data_augmentation=tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal",input_shape=(128,128,3)),
    tf.keras.layers.RandomRotation(0.1),
    tf.keras.layers.RandomZoom(0.1)
])
# Creating the model
model=tf.keras.Sequential([
    data_augmentation,
    tf.keras.layers.Rescaling(1.0/255,input_shape=(128,128,3)),
    tf.keras.layers.Conv2D(32,3,activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(32,3,activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(64,3,activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128,activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(num_classes,activation='softmax')
])

# Compiling the model.
model.compile(
    optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=["accuracy"]
)

#fitting the model
model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=12
)

```

```

Epoch 1/12
325/325 — 36s 109ms/step — accuracy: 0.3398 — loss: 1.5885 — val_accuracy: 0.4392 — val_loss: 1.4246
Epoch 2/12
325/325 — 36s 109ms/step — accuracy: 0.4327 — loss: 1.4346 — val_accuracy: 0.4221 — val_loss: 1.4891
Epoch 3/12
325/325 — 35s 106ms/step — accuracy: 0.4673 — loss: 1.3689 — val_accuracy: 0.4885 — val_loss: 1.3181
Epoch 4/12
325/325 — 34s 105ms/step — accuracy: 0.4916 — loss: 1.3284 — val_accuracy: 0.5088 — val_loss: 1.3087
Epoch 5/12
325/325 — 36s 109ms/step — accuracy: 0.5109 — loss: 1.2948 — val_accuracy: 0.4924 — val_loss: 1.3570
Epoch 6/12
325/325 — 34s 105ms/step — accuracy: 0.5217 — loss: 1.2665 — val_accuracy: 0.5460 — val_loss: 1.2250
Epoch 7/12
325/325 — 34s 105ms/step — accuracy: 0.5407 — loss: 1.2288 — val_accuracy: 0.5510 — val_loss: 1.2114
Epoch 8/12
325/325 — 34s 105ms/step — accuracy: 0.5469 — loss: 1.2099 — val_accuracy: 0.5640 — val_loss: 1.2137
Epoch 9/12
325/325 — 34s 105ms/step — accuracy: 0.5619 — loss: 1.1833 — val_accuracy: 0.5671 — val_loss: 1.2086
Epoch 10/12
325/325 — 34s 105ms/step — accuracy: 0.5713 — loss: 1.1662 — val_accuracy: 0.5617 — val_loss: 1.2086
Epoch 11/12
325/325 — 35s 108ms/step — accuracy: 0.5809 — loss: 1.1365 — val_accuracy: 0.5876 — val_loss: 1.1473
Epoch 12/12
325/325 — 35s 106ms/step — accuracy: 0.5874 — loss: 1.1298 — val_accuracy: 0.5873 — val_loss: 1.1498

```

Experiment 5 — Increased CNN Filters

Goal:

To test whether increasing the number of filters improves model performance.

Changes:

- **3 Conv2D layers: 64 → 64 → 128**
- **Dense layer: 128 neurons**

- **12 epochs**
- **No Dropout**
- **No Data Augmentation**
- **No Batch Normalization**

Results:

- **Final Training Accuracy: 96.01%**
- **Final Validation Accuracy: 65.07%**
- **Best Validation Accuracy: 66.41% (Epoch 9)**

Observation:

Validation accuracy was lower than the baseline, while training accuracy remained high.

```
[74]: num_classes = 6
# Creating the model
model=tf.keras.Sequential([
    tf.keras.layers.Rescaling(1.0/255,input_shape=(128,128,3)),
    tf.keras.layers.Conv2D(64,3,activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(64,3,activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(128,3,activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128,activation='relu'),
    tf.keras.layers.Dense(num_classes,activation='softmax')
])

[75]: # Compiling the model.
model.compile(
    optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=["accuracy"]
)

[*]: #fitting the model
model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=12
)
```

Epoch	Time	Step	acc	loss	val_acc	val_loss
Epoch 1/12	325s	233ms/step	0.3984	1.5103	0.4971	1.3442
Epoch 2/12	325s	228ms/step	0.5205	1.2646	0.5330	1.2465
Epoch 3/12	325s	224ms/step	0.5898	1.1051	0.5806	1.1898
Epoch 4/12	325s	234ms/step	0.6596	0.9366	0.5995	1.1901
Epoch 5/12	325s	251ms/step	0.7346	0.7455	0.6307	1.1861
Epoch 6/12	325s	256ms/step	0.7998	0.5695	0.6354	1.3876
Epoch 7/12	325s	235ms/step	0.8518	0.4229	0.6531	1.5398
Epoch 8/12	325s	230ms/step	0.8961	0.2963	0.6515	1.7170
Epoch 9/12	325s	229ms/step	0.9238	0.2222	0.6641	1.8089
Epoch 10/12	325s	228ms/step	0.9361	0.1836	0.6352	2.1905
Epoch 11/12	325s	228ms/step	0.9498	0.1488	0.6502	2.1716
Epoch 12/12	325s	233ms/step	0.9601	0.1251	0.6507	2.3841

Experiment 6 — Batch Normalization

Goal:

To test whether Batch Normalization improves model performance.

Changes:

- ***Added Batch Normalization after each Conv2D layer***
- ***3 Conv2D layers: 32 → 32 → 64***
- ***Dense layer: 128 neurons***
- ***12 epochs***
- ***No Dropout***
- ***No Data Augmentation***

Results:

- ***Final Training Accuracy: 96.20%***
- ***Final Validation Accuracy: 61.90%***
- ***Best Validation Accuracy: 66.50% (Epoch 11)***

Observation:

Batch Normalization did not improve the best validation accuracy compared to the baseline.

```

num_classes = 6
# Creating the model
model=tf.keras.Sequential([
    tf.keras.layers.Rescaling(1.0/255,input_shape=(128,128,3)),
    tf.keras.layers.Conv2D(32,3,activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(32,3,activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(64,3,activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128,activation='relu'),
    tf.keras.layers.Dense(num_classes,activation='softmax')
])

```

```

# Compiling the model.
model.compile(
    optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=["accuracy"]
)

```

```

#fitting the model
model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=12
)

```

```

Epoch 1/12
325/325 — 54s 165ms/step — accuracy: 0.4467 — loss: 1.6242 — val_accuracy: 0.2849 — val_loss: 2.0319
Epoch 2/12
325/325 — 55s 169ms/step — accuracy: 0.5773 — loss: 1.1442 — val_accuracy: 0.4454 — val_loss: 1.8494
Epoch 3/12
325/325 — 53s 162ms/step — accuracy: 0.6444 — loss: 0.9696 — val_accuracy: 0.4897 — val_loss: 1.5123
Epoch 4/12
325/325 — 53s 162ms/step — accuracy: 0.7104 — loss: 0.8046 — val_accuracy: 0.5478 — val_loss: 1.2650
Epoch 5/12
325/325 — 53s 162ms/step — accuracy: 0.7772 — loss: 0.6303 — val_accuracy: 0.6123 — val_loss: 1.1626
Epoch 6/12
325/325 — 52s 161ms/step — accuracy: 0.8310 — loss: 0.4715 — val_accuracy: 0.5669 — val_loss: 1.5791
Epoch 7/12
325/325 — 52s 161ms/step — accuracy: 0.8805 — loss: 0.3353 — val_accuracy: 0.5968 — val_loss: 1.7190
Epoch 8/12
325/325 — 53s 162ms/step — accuracy: 0.9107 — loss: 0.2488 — val_accuracy: 0.5106 — val_loss: 2.8629
Epoch 9/12
325/325 — 54s 165ms/step — accuracy: 0.9252 — loss: 0.2147 — val_accuracy: 0.6221 — val_loss: 1.7621
Epoch 10/12
325/325 — 55s 168ms/step — accuracy: 0.9420 — loss: 0.1729 — val_accuracy: 0.6069 — val_loss: 1.9348
Epoch 11/12
325/325 — 53s 164ms/step — accuracy: 0.9572 — loss: 0.1265 — val_accuracy: 0.6650 — val_loss: 1.7756
Epoch 12/12
325/325 — 54s 166ms/step — accuracy: 0.9620 — loss: 0.1143 — val_accuracy: 0.6190 — val_loss: 2.3198
[79]: <keras.src.callbacks.history.History at 0x13f30a090>

```

Experiment 7 — Padding = "same"

Goal:

To test whether using **padding="same"** improves model performance.

Changes:

- Added **padding="same"** to all Conv2D layers
- 3 Conv2D layers: 32 → 32 → 64
- Dense layer: 128 neurons
- 12 epochs

- **No Dropout**
- **No Data Augmentation**
- **No Batch Normalization**

Results:

- **Final Training Accuracy: 96.94%**
- **Final Validation Accuracy: 66.41%**
- **Best Validation Accuracy: 67.00% (Epoch 11)**

Observation:

The best validation accuracy was slightly lower than the baseline (67.42%).

```
30]: num_classes = 6
# Creating the model
model=tf.keras.Sequential([
    tf.keras.layers.Rescaling(1.0/255,input_shape=(128,128,3)),
    tf.keras.layers.Conv2D(32,3,activation='relu', padding='same'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(32,3,activation='relu', padding='same'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(64,3,activation='relu', padding='same'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128,activation='relu'),
    tf.keras.layers.Dense(num_classes,activation='softmax')
])

31]: # Compiling the model.
model.compile(
    optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=["accuracy"]
)

[*]: #fitting the model
model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=12
)

Epoch 1/12
325/325 ————— 37s 112ms/step - accuracy: 0.4164 - loss: 1.4690 - val_accuracy: 0.4960 - val_loss: 1.3515
Epoch 2/12
325/325 ————— 38s 115ms/step - accuracy: 0.5427 - loss: 1.2109 - val_accuracy: 0.5528 - val_loss: 1.2178
Epoch 3/12
325/325 ————— 36s 111ms/step - accuracy: 0.6179 - loss: 1.0426 - val_accuracy: 0.6015 - val_loss: 1.1414
Epoch 4/12
325/325 ————— 37s 113ms/step - accuracy: 0.6933 - loss: 0.8531 - val_accuracy: 0.6338 - val_loss: 1.1169
Epoch 5/12
325/325 ————— 38s 117ms/step - accuracy: 0.7549 - loss: 0.6839 - val_accuracy: 0.6089 - val_loss: 1.2877
Epoch 6/12
325/325 ————— 37s 112ms/step - accuracy: 0.8214 - loss: 0.5057 - val_accuracy: 0.6493 - val_loss: 1.3000
Epoch 7/12
325/325 ————— 36s 110ms/step - accuracy: 0.8739 - loss: 0.3726 - val_accuracy: 0.6567 - val_loss: 1.4639
Epoch 8/12
325/325 ————— 36s 110ms/step - accuracy: 0.9068 - loss: 0.2757 - val_accuracy: 0.6515 - val_loss: 1.7045
Epoch 9/12
325/325 ————— 36s 111ms/step - accuracy: 0.9367 - loss: 0.1947 - val_accuracy: 0.6412 - val_loss: 1.9782
Epoch 10/12
325/325 ————— 36s 110ms/step - accuracy: 0.9460 - loss: 0.1608 - val_accuracy: 0.6632 - val_loss: 2.0923
Epoch 11/12
325/325 ————— 36s 110ms/step - accuracy: 0.9606 - loss: 0.1195 - val_accuracy: 0.6700 - val_loss: 2.2951
Epoch 12/12
325/325 ————— 36s 109ms/step - accuracy: 0.9694 - loss: 0.0984 - val_accuracy: 0.6641 - val_loss: 2.2973
```

Experiment 8 — Combined Model

Goal:

To test whether combining the most promising changes from the previous experiments could improve validation accuracy.

Why we are doing this:

The previous experiments tested different techniques individually. Some changes produced results close to the baseline, so we combined them into one model to see whether their effects would work together and improve the overall performance.

Changes:

- 3 Conv2D layers: 32 → 32 → 64
- Added `padding="same"`
- Added Batch Normalization
- Added Dropout (0.5)
- Dense layer: 128 neurons
- No Data Augmentation
- Trained for 20 epochs

Results:

- Final Training Accuracy: 71.24%
- Final Validation Accuracy: 65.24%
- Best Validation Accuracy: 65.24% (Epoch 20)

Observation:

Combining the changes did not improve the model. The best validation accuracy was lower than the baseline model's 67.42%. This shows that changes that perform reasonably well individually do not necessarily improve performance when combined.

Conclusion:

The original Basic CNN remains the best-performing model so far, with a best validation accuracy of 67.42%. The combined model also showed lower training accuracy, suggesting that the added regularization techniques may have reduced the model's ability to learn the training data.

```
[85]: num_classes = 6

model = tf.keras.Sequential([
    tf.keras.layers.Rescaling(1.0/255, input_shape=(128,128,3)),
    tf.keras.layers.Conv2D(32, 3, activation='relu', padding='same'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(32, 3, activation='relu', padding='same'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(64, 3, activation='relu', padding='same'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(num_classes, activation='softmax')
])

[86]: # Compiling the model.
model.compile(
    optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=['accuracy']
)
```

```
Epoch 1/20
325/325 — 62s 189ms/step — accuracy: 0.2832 — loss: 1.8253 — val_accuracy: 0.2483 — val_loss: 2.5572
Epoch 2/20
325/325 — 61s 186ms/step — accuracy: 0.3406 — loss: 1.5736 — val_accuracy: 0.3498 — val_loss: 1.7142
Epoch 3/20
325/325 — 60s 184ms/step — accuracy: 0.3772 — loss: 1.5136 — val_accuracy: 0.4313 — val_loss: 1.4829
Epoch 4/20
325/325 — 59s 180ms/step — accuracy: 0.4035 — loss: 1.4482 — val_accuracy: 0.4342 — val_loss: 1.4486
Epoch 5/20
325/325 — 58s 179ms/step — accuracy: 0.4294 — loss: 1.3959 — val_accuracy: 0.4475 — val_loss: 1.4540
Epoch 6/20
325/325 — 59s 182ms/step — accuracy: 0.4523 — loss: 1.3606 — val_accuracy: 0.4362 — val_loss: 1.4712
Epoch 7/20
325/325 — 58s 179ms/step — accuracy: 0.4682 — loss: 1.3134 — val_accuracy: 0.4850 — val_loss: 1.3663
Epoch 8/20
325/325 — 58s 179ms/step — accuracy: 0.4963 — loss: 1.2517 — val_accuracy: 0.5537 — val_loss: 1.2173
Epoch 9/20
325/325 — 58s 179ms/step — accuracy: 0.5285 — loss: 1.1848 — val_accuracy: 0.5099 — val_loss: 1.2742
Epoch 10/20
325/325 — 60s 183ms/step — accuracy: 0.5517 — loss: 1.1409 — val_accuracy: 0.5296 — val_loss: 1.3330
Epoch 11/20
325/325 — 59s 181ms/step — accuracy: 0.5753 — loss: 1.0674 — val_accuracy: 0.4259 — val_loss: 1.5667
Epoch 12/20
325/325 — 58s 179ms/step — accuracy: 0.6019 — loss: 1.0076 — val_accuracy: 0.5707 — val_loss: 1.1402
Epoch 13/20
325/325 — 59s 180ms/step — accuracy: 0.6135 — loss: 0.9652 — val_accuracy: 0.5613 — val_loss: 1.2189
Epoch 14/20
325/325 — 60s 184ms/step — accuracy: 0.6298 — loss: 0.9238 — val_accuracy: 0.5694 — val_loss: 1.2755
Epoch 15/20
325/325 — 59s 181ms/step — accuracy: 0.6420 — loss: 0.9009 — val_accuracy: 0.5260 — val_loss: 1.3690
Epoch 16/20
325/325 — 58s 179ms/step — accuracy: 0.6618 — loss: 0.8496 — val_accuracy: 0.6426 — val_loss: 1.1047
Epoch 17/20
325/325 — 57s 175ms/step — accuracy: 0.6739 — loss: 0.8180 — val_accuracy: 0.5406 — val_loss: 1.2185
Epoch 18/20
325/325 — 59s 180ms/step — accuracy: 0.6906 — loss: 0.7735 — val_accuracy: 0.6464 — val_loss: 1.1442
Epoch 19/20
325/325 — 59s 182ms/step — accuracy: 0.6995 — loss: 0.7479 — val_accuracy: 0.6210 — val_loss: 1.1653
Epoch 20/20
325/325 — 58s 180ms/step — accuracy: 0.7124 — loss: 0.7210 — val_accuracy: 0.6524 — val_loss: 1.0594
```

Experiment 9 —CNN with Adjusted Learning Rate and Early Stopping.

Goal:

To further improve the CNN performance by adjusting the learning rate and using Early Stopping during training.

Changes:

- **Used the CNN architecture from the previous experiment**
- **Adam optimizer with a learning rate of 0.0005**
- **Added Early Stopping**
- **Early Stopping monitored validation accuracy**
- **Maximum of 20 epochs**

```
num_classes = 6

model = tf.keras.Sequential([
    tf.keras.layers.Rescaling(1.0/255, input_shape=(128,128,3)),
    tf.keras.layers.Conv2D(32, 3, activation='relu', padding='same'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(32, 3, activation='relu', padding='same'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(64, 3, activation='relu', padding='same'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(num_classes, activation='softmax')
])

# Compiling the model.
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0005),
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=["accuracy"]
)

early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor='val_accuracy',
    patience=4,
    restore_best_weights=True
)

model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=20,
    callbacks=[early_stopping]
)
```

```
Epoch 1/20
325/325 — 58s 178ms/step — accuracy: 0.3301 — loss: 1.7007 — val_accuracy: 0.2241 — val_loss: 2.2406
Epoch 2/20
325/325 — 59s 181ms/step — accuracy: 0.3851 — loss: 1.5166 — val_accuracy: 0.4722 — val_loss: 1.4057
Epoch 3/20
325/325 — 59s 183ms/step — accuracy: 0.4231 — loss: 1.4347 — val_accuracy: 0.4899 — val_loss: 1.3624
Epoch 4/20
325/325 — 60s 185ms/step — accuracy: 0.4533 — loss: 1.3789 — val_accuracy: 0.4551 — val_loss: 1.3809
Epoch 5/20
325/325 — 59s 182ms/step — accuracy: 0.4835 — loss: 1.3137 — val_accuracy: 0.5065 — val_loss: 1.3086
Epoch 6/20
325/325 — 60s 185ms/step — accuracy: 0.5157 — loss: 1.2423 — val_accuracy: 0.5608 — val_loss: 1.2352
Epoch 7/20
325/325 — 60s 185ms/step — accuracy: 0.5443 — loss: 1.1715 — val_accuracy: 0.5716 — val_loss: 1.1956
Epoch 8/20
325/325 — 59s 182ms/step — accuracy: 0.5720 — loss: 1.1073 — val_accuracy: 0.5635 — val_loss: 1.1830
Epoch 9/20
325/325 — 59s 182ms/step — accuracy: 0.6031 — loss: 1.0278 — val_accuracy: 0.5649 — val_loss: 1.2072
Epoch 10/20
325/325 — 60s 183ms/step — accuracy: 0.6256 — loss: 0.9690 — val_accuracy: 0.5629 — val_loss: 1.2123
Epoch 11/20
325/325 — 58s 179ms/step — accuracy: 0.6528 — loss: 0.8948 — val_accuracy: 0.6437 — val_loss: 1.1243
Epoch 12/20
325/325 — 60s 184ms/step — accuracy: 0.6682 — loss: 0.8508 — val_accuracy: 0.6632 — val_loss: 1.0428
Epoch 13/20
325/325 — 58s 178ms/step — accuracy: 0.6943 — loss: 0.7815 — val_accuracy: 0.6626 — val_loss: 1.0543
Epoch 14/20
325/325 — 58s 180ms/step — accuracy: 0.7102 — loss: 0.7383 — val_accuracy: 0.5552 — val_loss: 1.2212
Epoch 15/20
325/325 — 60s 185ms/step — accuracy: 0.7253 — loss: 0.6942 — val_accuracy: 0.6605 — val_loss: 1.0763
Epoch 16/20
325/325 — 60s 184ms/step — accuracy: 0.7417 — loss: 0.6648 — val_accuracy: 0.6632 — val_loss: 1.0722
[0]: <keras.src.callbacks.history.History at 0x1436193d0>
```

Stage 2: Error Analysis

→ Plastic was the weakest class; paper/metal/plastic showed substantial confusion.

```
[102]: import numpy as np
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt

y_true = []
y_pred = []

for images, labels in val_ds:
    predictions = model.predict(images, verbose=0)
    y_true.extend(labels.numpy())
    y_pred.extend(np.argmax(predictions, axis=1))

y_true = np.array(y_true)
y_pred = np.array(y_pred)

2026-08-29 11:55:09.660269: W tensorflow/core/framework/local_rendevous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End
of sequence

[103]: cm = confusion_matrix(y_true, y_pred)

print(cm)

[[506  45  28  76  23  45]
 [ 16 628  28  17  39  22]
 [  6  76 503  38  83  39]
 [ 16  61 112 436  62  57]
 [ 11 109  94  73 381  79]
 [ 19  60  48  61  57 500]]
```

```
[105]: from sklearn.metrics import classification_report

print(classification_report(
    y_true,
    y_pred,
    target_names=['cardboard', 'glass', 'metal', 'paper', 'plastic', 'trash']
))
```

	precision	recall	f1-score	support
cardboard	0.88	0.70	0.78	723
glass	0.64	0.84	0.73	750
metal	0.62	0.68	0.65	745
paper	0.62	0.59	0.60	744
plastic	0.59	0.51	0.55	747
trash	0.67	0.67	0.67	745
accuracy			0.66	4454
macro avg	0.67	0.66	0.66	4454
weighted avg	0.67	0.66	0.66	4454

Stage 3: Transfer Learning

→ Tested a pretrained MobileNetV2 model with ImageNet weights to determine whether pretrained visual features could improve waste classification.

→ Initially froze the MobileNetV2 base and trained a new classifier for the six waste categories.

→ Best validation accuracy: 57.09%, which was lower than CNN's 67.42%.

→ Next, fine-tune the layers of MobileNetV2 with a low learning rate so the pretrained features can adapt specifically to the waste dataset.

```
0]: num_classes = 6

base_model = tf.keras.applications.MobileNetV2(
    input_shape=(128, 128, 3),
    include_top=False,
    weights='imagenet'
)

base_model.trainable = False

1]: model = tf.keras.Sequential([
    tf.keras.layers.Rescaling(1./127.5, offset=-1),
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(num_classes, activation='softmax')
])
model.compile(
    optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=['accuracy']
)
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=15
)
```

Epoch 1/15
325/325 ————— 29s 86ms/step - accuracy: 0.4994 - loss: 1.3035 - val_accuracy: 0.5200 - val_loss: 1.2699
Epoch 2/15
325/325 ————— 27s 84ms/step - accuracy: 0.5191 - loss: 1.2574 - val_accuracy: 0.5292 - val_loss: 1.2471
Epoch 3/15
325/325 ————— 28s 85ms/step - accuracy: 0.5401 - loss: 1.2066 - val_accuracy: 0.5317 - val_loss: 1.2434
Epoch 4/15
325/325 ————— 28s 87ms/step - accuracy: 0.5532 - loss: 1.1853 - val_accuracy: 0.5366 - val_loss: 1.2249
Epoch 5/15
325/325 ————— 28s 85ms/step - accuracy: 0.5649 - loss: 1.1530 - val_accuracy: 0.5519 - val_loss: 1.2082
Epoch 6/15
325/325 ————— 27s 82ms/step - accuracy: 0.5742 - loss: 1.1257 - val_accuracy: 0.5507 - val_loss: 1.1886
Epoch 7/15
325/325 ————— 27s 82ms/step - accuracy: 0.5851 - loss: 1.0988 - val_accuracy: 0.5539 - val_loss: 1.1935
Epoch 8/15
325/325 ————— 27s 82ms/step - accuracy: 0.5946 - loss: 1.0728 - val_accuracy: 0.5588 - val_loss: 1.1858
Epoch 9/15
325/325 ————— 28s 85ms/step - accuracy: 0.6013 - loss: 1.0473 - val_accuracy: 0.5519 - val_loss: 1.2069
Epoch 10/15
325/325 ————— 28s 85ms/step - accuracy: 0.6133 - loss: 1.0235 - val_accuracy: 0.5615 - val_loss: 1.1880
Epoch 11/15
325/325 ————— 28s 85ms/step - accuracy: 0.6171 - loss: 1.0143 - val_accuracy: 0.5667 - val_loss: 1.1883
Epoch 12/15
325/325 ————— 29s 88ms/step - accuracy: 0.6276 - loss: 0.9878 - val_accuracy: 0.5700 - val_loss: 1.1834
Epoch 13/15
325/325 ————— 28s 85ms/step - accuracy: 0.6379 - loss: 0.9695 - val_accuracy: 0.5678 - val_loss: 1.1965
Epoch 14/15
325/325 ————— 28s 87ms/step - accuracy: 0.6414 - loss: 0.9523 - val_accuracy: 0.5602 - val_loss: 1.2206
Epoch 15/15
325/325 ————— 27s 83ms/step - accuracy: 0.6488 - loss: 0.9334 - val_accuracy: 0.5709 - val_loss: 1.2052

Stage 4: Fine-Tuning MobileNetV2

Goal

To improve the validation accuracy achieved by the previous CNN and frozen transfer-learning models by fine-tuning the pretrained MobileNetV2 model on our waste-classification dataset.

Why We Are Doing This

The scratch CNN achieved approximately 67.42% validation accuracy, while the frozen MobileNetV2 achieved approximately 57.09%. Although the pretrained model already contained useful visual features, keeping the entire base model frozen prevented those features from adapting fully to the specific characteristics of our waste images.

Therefore, we decided to fully fine-tune MobileNetV2 so that its pretrained features could be adjusted to better recognize our six waste categories.

Changes

- Used MobileNetV2 pretrained on ImageNet
- `include_top=False` to remove the original ImageNet classifier
- Fully fine-tuned the MobileNetV2 base model
- Used `GlobalAveragePooling2D`
- Added a Dense layer with 128 neurons
- Added Dropout (0.3)
- Added a final 6-class softmax output layer
- Used a small Adam learning rate of $1e-5$
- Trained for a maximum of 20 epochs
- Used Early Stopping with `patience=3`
- Used `restore_best_weights=True`

Why These Changes Were Selected

- **MobileNetV2:** Provides strong pretrained visual features that can be adapted to our waste dataset.
- **Full fine-tuning:** Allows the pretrained convolutional features to learn patterns specific to cardboard, glass, metal, paper, plastic, and trash.
- **Global Average Pooling:** Reduces the feature maps to a compact representation before classification and avoids the very large number of parameters that would result from flattening the feature maps.
- **Dropout (0.3):** Helps reduce overfitting while training the classifier.
- **Learning rate = $1e-5$:** A small learning rate helps make gradual changes to the pretrained weights instead of changing them too aggressively.
- **Early Stopping:** Prevents unnecessary training when validation loss stops improving and restores the best-performing weights.

Training Results

The model showed a substantial improvement compared with our previous experiments.

- Epoch 1 validation accuracy: 63.88%
- Epoch 10 validation accuracy: 85.20%
- Epoch 17 validation accuracy: 87.76%
- Best validation loss: 0.3911 at Epoch 14
- Training automatically stopped at Epoch 17 because validation loss did not improve for three consecutive epochs.

The model achieved approximately 87% validation accuracy, substantially exceeding the previous scratch CNN result of 67.42%.

Conclusion

Full fine-tuning of MobileNetV2 produced a significant improvement over both the scratch CNN and the frozen transfer-learning model. This suggests that allowing the pretrained features to adapt to our specific waste images was much more effective than keeping the pretrained feature extractor completely frozen.

Next Step

Evaluate the selected model on the independent test dataset to obtain its final test accuracy. After that, generate a confusion matrix and classification report to analyze the model's performance for each waste category.

```
[115]: num_classes = 6

base_model = tf.keras.applications.MobileNetV2(
    input_shape=(128, 128, 3),
    include_top=False,
    weights='imagenet'
)

base_model.trainable = True

[116]: model = tf.keras.Sequential([
    tf.keras.layers.Rescaling(1./127.5, offset=-1),
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(num_classes, activation='softmax')
])
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=['accuracy']
)
early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)
history_finetune = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=20,
    callbacks=[early_stopping]
)
```

```
Epoch 9/20
325/325 — 123s 379ms/step — accuracy: 0.8841 — loss: 0.3488 — val_accuracy: 0.8485 — val_loss: 0.4406
Epoch 10/20
325/325 — 124s 382ms/step — accuracy: 0.8949 — loss: 0.3186 — val_accuracy: 0.8520 — val_loss: 0.4265
Epoch 11/20
325/325 — 125s 385ms/step — accuracy: 0.9047 — loss: 0.2832 — val_accuracy: 0.8612 — val_loss: 0.4137
Epoch 12/20
325/325 — 123s 377ms/step — accuracy: 0.9154 — loss: 0.2551 — val_accuracy: 0.8615 — val_loss: 0.4049
Epoch 13/20
325/325 — 123s 377ms/step — accuracy: 0.9261 — loss: 0.2282 — val_accuracy: 0.8705 — val_loss: 0.3969
Epoch 14/20
325/325 — 121s 373ms/step — accuracy: 0.9332 — loss: 0.2050 — val_accuracy: 0.8714 — val_loss: 0.3911
Epoch 15/20
325/325 — 123s 379ms/step — accuracy: 0.9430 — loss: 0.1788 — val_accuracy: 0.8731 — val_loss: 0.3924
Epoch 16/20
325/325 — 120s 368ms/step — accuracy: 0.9488 — loss: 0.1566 — val_accuracy: 0.8752 — val_loss: 0.3917
Epoch 17/20
325/325 — 937s 3s/step — accuracy: 0.9544 — loss: 0.1459 — val_accuracy: 0.8776 — val_loss: 0.3943
```

Stage 5: Final Model Evaluation

Goal

To evaluate the final fine-tuned MobileNetV2 model on a separate test dataset that was not used during training or model selection.

Why We Are Doing This

Validation accuracy was used during model development to monitor performance and select the best model. However, the validation set was part of the development process. Therefore, we use a separate **test dataset** to measure how well the final model performs on previously unseen images.

Testing Process

The final model was evaluated using:

- **Test images:** 4,448
- **Number of classes:** 6
- **Shuffle:** Disabled to maintain consistent ordering during evaluation
- **Training:** No further training was performed on the test dataset

The model was evaluated using:

```
test_loss, test_accuracy = model.evaluate(test_ds)
```

Final Test Results

- **Test Accuracy:** 88.33%
- **Test Loss:** 0.3794
- **Test Images:** 4,448



Conclusion

The fine-tuned MobileNetV2 achieved a final **test accuracy of 88.33%** on 4,448 previously unseen images. This substantially exceeded the performance of our original scratch CNN.

The results demonstrate that fine-tuning a pretrained model allowed the network to learn features that were better suited to our waste-classification dataset. Since the test dataset was

kept separate from the training and validation process, the test accuracy provides an estimate of the model's performance on previously unseen images from the same dataset.

```
: #Testing the model on test_data
test_ds = tf.keras.utils.image_dataset_from_directory(
    os.path.join(DATA_DIR, 'test'),
    image_size=IMAGE_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=False
)

test_loss, test_accuracy = model.evaluate(test_ds)

print("Test Accuracy:", test_accuracy)
print("Test Loss:", test_loss)
```

Found 4448 files belonging to 6 classes.

70/70  5s 67ms/step - accuracy: 0.8833 - loss: 0.3794

Test Accuracy: 0.8833183646202087

Test Loss: 0.3793914020061493

Confusion Matrix and Classification report:

Confusion Matrix:

```
[[672  2  11  17  10  13]
 [  0 698  17  3  26  7]
 [  7  9 678  24  10  11]
 [ 37  4  16 631  31  25]
 [  5 39  26  46 602  29]
 [ 15  7  16  23  33 648]]
```

Classification Report:

	precision	recall	f1-score	support
cardboard	0.91	0.93	0.92	725
glass	0.92	0.93	0.92	751
metal	0.89	0.92	0.90	739
paper	0.85	0.85	0.85	744
plastic	0.85	0.81	0.83	747
trash	0.88	0.87	0.88	742
accuracy			0.88	4448
macro avg	0.88	0.88	0.88	4448
weighted avg	0.88	0.88	0.88	4448

Stage 6: Raspberry Pi Deployment.

Goal

To deploy the trained MobileNetV2 model on a Raspberry Pi and build a working waste-sorting system that can classify waste and automatically direct it to the appropriate bin.

System Overview

The trained model was integrated with a Raspberry Pi camera, push button, and servo motor to create the physical waste-sorting system.

The system follows this process:

1. The Raspberry Pi starts the camera preview.
2. The user positions the waste item in front of the camera.
3. The user presses the button to capture an image.
4. The captured image is processed and resized to **128 × 128 pixels**, matching the input size used during model training.

5. The MobileNetV2 model predicts one of the six waste categories:
 - Cardboard
 - Glass
 - Metal
 - Paper
 - Plastic
 - Trash
6. The prediction confidence is checked against a **60% confidence threshold**.
7. If the model predicts a recyclable category with at least 60% confidence, the servo moves the sorting mechanism toward the recycling bin.
8. If the model predicts Trash, or if the prediction confidence is below 60%, the item is directed to the trash bin.
9. The servo returns to its neutral position after sorting.

Model Input and Camera Preview

The model was trained using images resized to 128×128 pixels. However, using such a small resolution for the camera preview would make it difficult for the user to see and position the waste item.

Therefore, the camera preview and model input serve different purposes:

- Camera preview: Uses a larger resolution to make positioning the waste item easier.
- Model input: The captured image is processed and resized to 128×128 pixels before being passed to the model.

For image downscaling, `cv2.INTER_AREA` was used because it is designed for reducing image size and helps preserve the overall appearance of the image.

Six Classes and Two Physical Bins

Although the physical system contains only two bins, the model classifies waste into six categories.

The six-class prediction provides more specific information about the type of waste before the system decides which physical bin to use. The recyclable categories are mapped to the recycling bin, while trash is mapped to the trash bin.

Confidence Threshold

A 60% confidence threshold was added to make the physical sorting system more cautious when the model is uncertain.

If the model's highest prediction confidence is below 60%, the item is sent to the trash bin instead of being sorted based on an uncertain prediction.

This provides a simple way to handle low-confidence predictions in the physical system.

Hardware

The deployed system uses:

- Raspberry Pi
- Raspberry Pi camera
- Push button
- Servo motor
- Two physical bins
- Sorting mechanism

Result

The trained MobileNetV2 model was successfully integrated with the Raspberry Pi hardware to create a working prototype capable of capturing waste images, classifying them, and automatically moving the sorting mechanism based on the model prediction.